

Basics of MATLAB Programming

Chapter 3

Copyright © 2017 STEM Course Prep



OVERVIEW

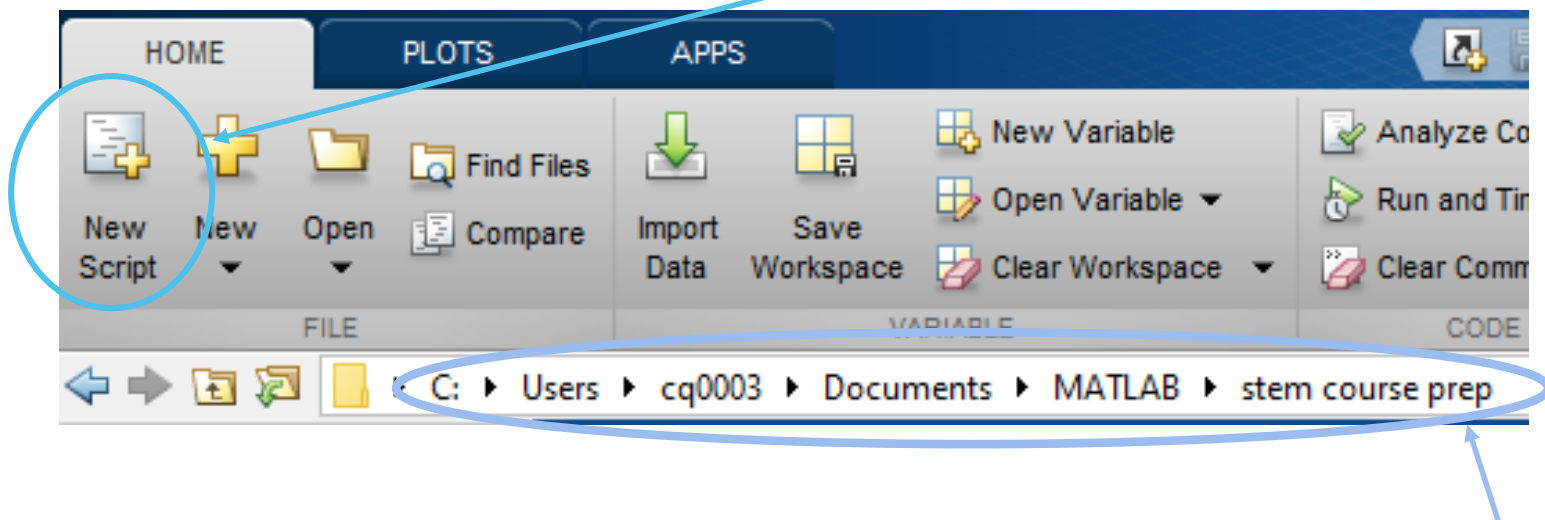
- In Chapter 3 you will learn about
 - Script files
 - Input/output methods
 - Basic plotting
 - Loading/saving data files
 - User-defined functions with one output
 - Simple programs

SCRIPT FILES

- Until now, we have been able to use the command window for calculations
- Command window is not the best method for longer problems or code we want to reuse later
- Script files are best suited for these applications
- Script files are defined by the user and are saved with the extension .m

SCRIPT FILES

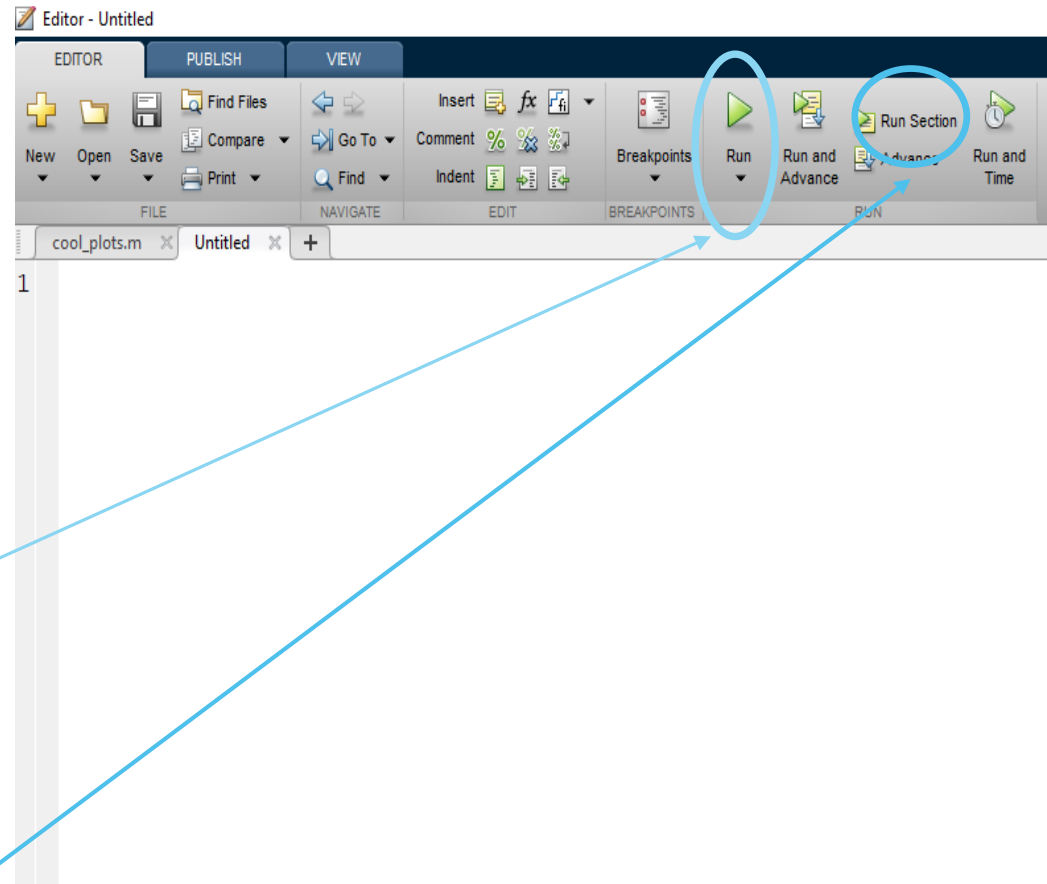
- To create a script file click New Script



- When saving the script file it saves by default in this location

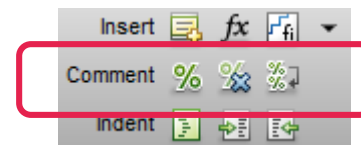
SCRIPT FILES

- ❑ Clicking New Script pops up a new window
- ❑ Click the Save button to save
- ❑ After typing code, run the script file by either
 - ❑ Hitting the Run button
 - ❑ Typing the script name in at the Command Window prompt
- ❑ Can put multiple codes in one file and run them separately by using %% and then hitting Run Section



COMMENTS

- Be kind to yourself and others – **document your code!!**
- **Comments** are used to describe what your script is doing
 - These descriptions are very important if your code will be used by others
- Create comments by using a %
 - Everything to the right of the % will be a comment and colored green
 - MATLAB ignores comments
- Easy to add/remove % using the comment options in the script file



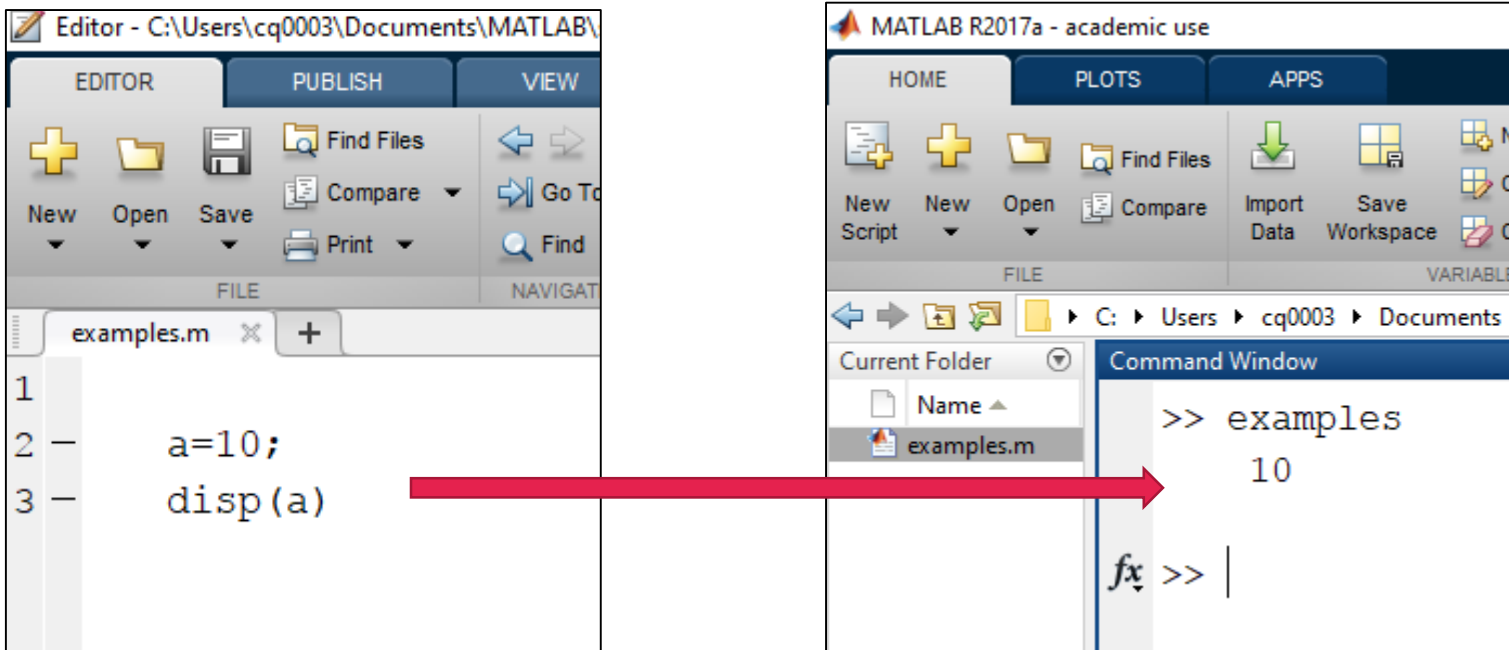
USER INPUT

- Think of an instance when a person may need to enter data in order for your code to run
 - Data needed could be salary, age, a color, number of data points, etc.
- **input** is the built-in function for this task
- **input** displays a prompt selected by you and then saves user's input in the variable specified
- Two options for inputs
 - Numbers
 - `name=input('prompt')`
 - Strings (text)
 - `name=input('prompt', 's')`

's' indicates a string will be input

disp

- Quick and easy way to display output
 - **disp** – displays data without any formatting
 - Matrices can easily be displayed with disp



fprintf

- Imagine you have a code and you want to display output that looks something like:
Water temperature is 127.456 degrees Fahrenheit
- **fprintf** allows formatted output to be displayed
- **fprintf** uses conversion characters and field width
 - Conversion characters indicate what type of output is being displayed
 - %d – integers
 - %f – floats
 - %s – strings
 - %c – characters

fprintf

- Field width is an integer that indicates how many spaces can be used by the output
 - To allow 10 spaces for the output, field width would =10
- Field width is combined with a conversion character to specify the desired format
 - Examples:
 - %5d - 5 spaces for an integer value
 - %7s – 7 spaces for a string
- Note: If the specified field width is smaller than the value/string the entire value/string is still displayed

fprintf

- Can specify # of digits past decimal point with floating point numbers
 - Examples:
 - %10.3f - field width of 10 spaces with 3 digits past the decimal
 - if a field width of 10 isn't needed blank spaces are inserted before the number
 - %.5f – field width is automatically chosen and there will be 5 digits past the decimal
 - I generally use this format since field width doesn't have to be specified

fprintf

- Additional ways to format output:
 - Left justify – use dash (-)
 - Example: %-10d – integer value will be left aligned over 10 spaces
 - Slash – use \\
 - Single quote – use ‘ ‘
 - Tab - \t same effect as hitting the tab key
 - Newline - \n same effect as hitting the enter key
- Now let's see how to put all this together

EXAMPLES OF **fprintf**

```
>>temp=127.45613;
```

```
>>fprintf('Water temperature is %.3f degrees\n',temp)
```

```
Water temperature is 127.456 degrees
```

Moves cursor
to next line



```
>>sales=20.47638;
```

```
>>day=5;
```

```
>>fprintf('Average sales on day %5d = %.2f\n',day,sales)
```

```
Average sales on day      5 = 20.48
```

Why are there blanks?



Value rounded b/c only
wanted 2 #s past decimal



EXAMPLES OF **fprintf**

```
>>str='i hate monday';  
>>fprintf('%-20s!\n',str)  
i hate monday      !
```

Left-aligned b/c used –
Default would be
right-aligned

7 blank spaces b/c string was
13 characters and we specified
a field width of 20

SCRIPTS

- Combining input and output in scripts is typical
- General format:
 - ❖ Use **input** to get data from user
 - ❖ Perform calculations
 - ❖ Use **fprintf** to display formatted data
- General practice is to use semi-colons (;) to suppress output
 - **fprintf** should be where output is displayed

EXAMPLE

- Basal metabolic rate(BMR) is used to determine daily calorie recommendations
- For male adults the calculation for BMR is:
$$66 + (6.23 \times \text{pounds body weight}) + (12.7 \times \text{height in inches}) - (6.76 \times \text{age}) = \text{BMR}^{**}$$
- For a male that exercises 3-5 times per week the daily caloric requirement is $1.55 \times \text{BMR}^{**}$
- Write a script that will collect data from the user and determine the required amount of calories/day
- For the ladies...
 - $655 + (4.35 \times \text{pounds body weight}) + (4.7 \times \text{height in inches}) - (4.7 \times \text{age}) = \text{BMR}^{**}$

^{**} <https://www.medicalnewstoday.com/articles/245588.php>

EXAMPLE ¹⁷

```
weight=input('enter weight (lbs):');  
height=input('enter height (in):');  
age=input('enter age:');  
  
BMR=66+6.23*weight+12.7*height-6.76*age;  
calories=1.55*BMR;  
fprintf('Daily caloric requirement = %.2f for a man  
of age %d\n',calories,age)
```

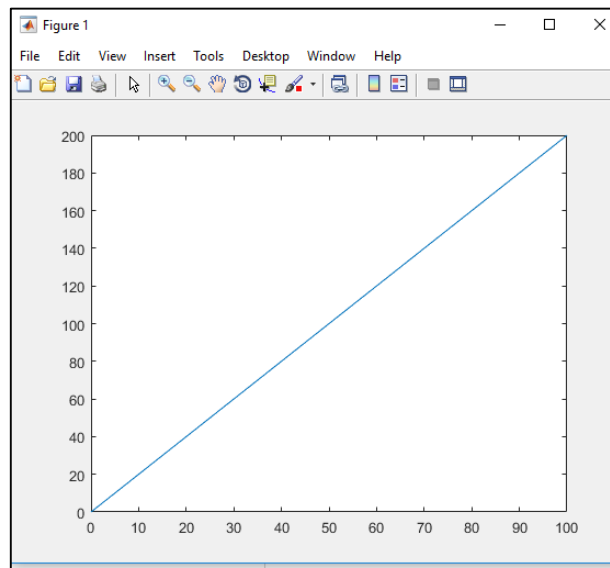
```
enter weight (lbs):200
```

```
enter height (in):72
```

```
enter age:23
```

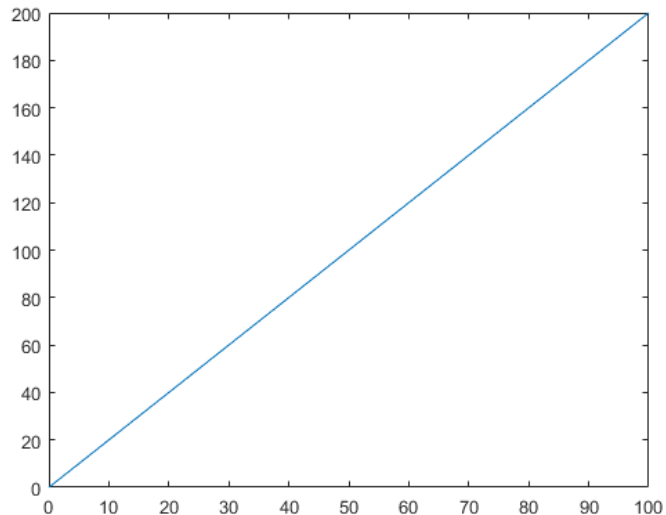
```
Daily caloric requirement = 3209.93 for a man of  
age 23
```

- Countless plots can be easily generated by MATLAB
- Will cover the basics of 2-D plotting
 - **plot** – built-in 2-D plot function
- Arrays of data (like x and y) are needed for plotting
 - Arrays must be the same length
 - Default plot will have a solid blue line
- A new window pops up with the plot
- Plots can be saved with any of the standard pic extensions

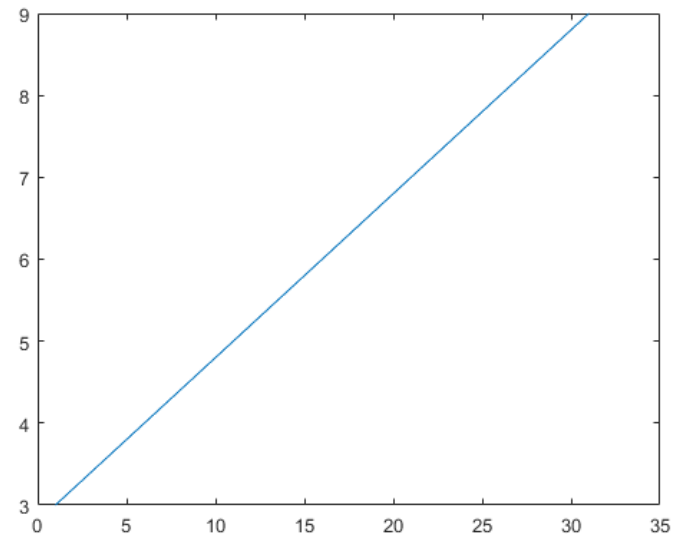


- **plot** – built-in function for 2-D plots
 - Use **plot(x,y)** to plot x and y arrays against each other
 - **plot(y)** can also be used if horizontal data increments by 1

```
t=linspace(0, 100, 100);  
v=2*t;  
plot(t,v)
```



```
t=0:30;  
v=3+.2*t;  
plot(v)
```



plot

- Many options exist for formatting plots
- Will cover:
 - Color
 - Data markers
 - Line types
 - Axes
 - Labels
 - Title
 - Legend
- Color, data markers and line type options are specified in a third input argument which is a string
 - `plot(t,v, 'c-.*')` – will have cyan-colored, dash-dot lines with * markers
 - Order of options doesn't matter

See **help plot** for list of options below

b	blue	.	point	-	solid
g	green	o	circle	:	dotted
r	red	x	x-mark	-.	dash-dot
c	cyan	+	plus	--	dashed
m	magenta	*	star	(none)	no line
y	yellow	s	square		
k	black	d	diamond		
w	white	v	triangle (down)		
		^	triangle (up)		
		<	triangle (left)		
		>	triangle (right)		
		p	pentagram		
		h	hexagram		

PLOT LABELS

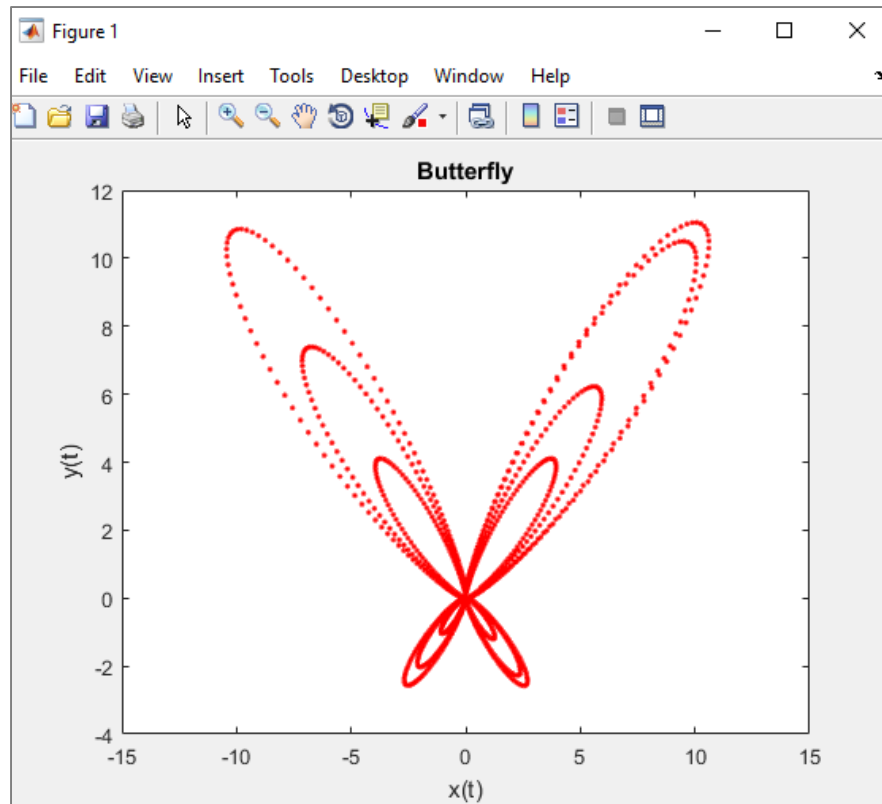
- Axis labels and titles must be specified
- Axis Labels:
 - `xlabel('string')`
 - `ylabel('string')`
- Title:
 - `title('string')`
- MATLAB will determine min/max axis values if none are specified
 - To specify values:
 - `axis([xmin xmax ymin ymax])`
 - `axis tight` – gets rid of excess white space on edges of plot
 - `axis equal` – fixes distortion issue for circular plots



Anything in quotes
gets displayed

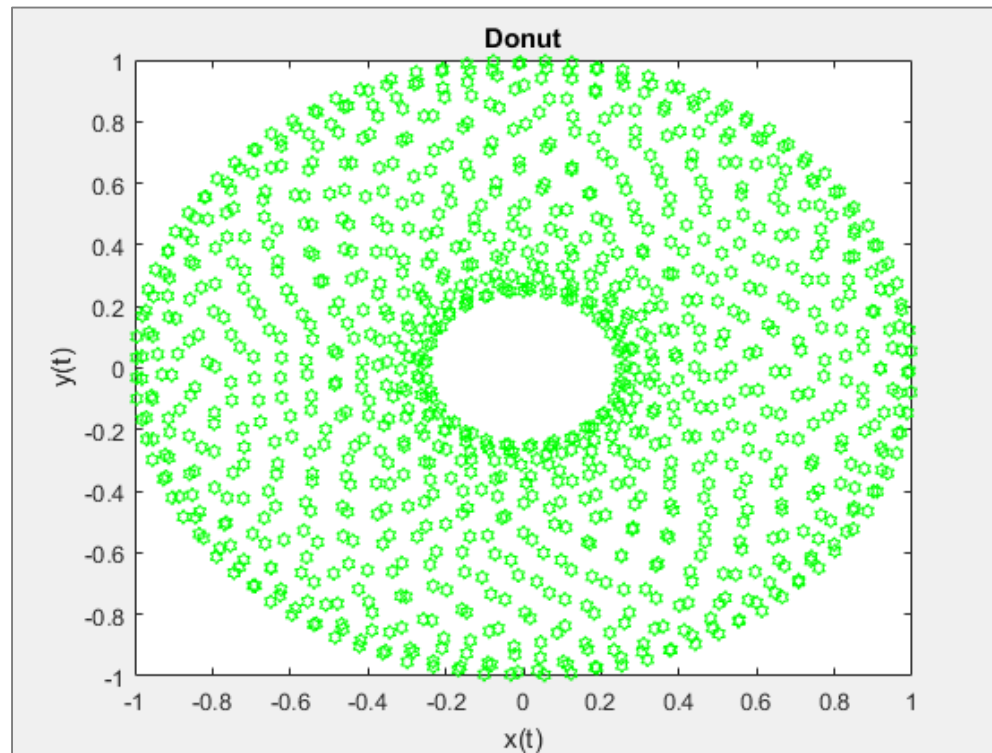
plot EXAMPLES ²³

```
t=0:.01:20;  
x=sin(t).*(exp(cos(t))-2*cos(4*t)-(sin(t/12).^5))  
y=cos(t).*(exp(cos(t))-2*cos(4*t)-(sin(t/12).^5))  
plot(x,y,'r.')  
xlabel('x(t)')  
ylabel('y(t)')  
title('Butterfly')
```



plot EXAMPLES ²⁴

```
t=0:.1:32*pi;  
x=sin(t).*cos(-7*t/16)-.25*cos(t).*sin(-7*t/16);  
y=sin(t).*sin(-7*t/16)+.25*cos(t).*cos(-7*t/16);  
plot(x,y,'gh')  
xlabel('x(t)')  
ylabel('y(t)')  
title('Donut')
```



MORE PLOT FUNCTIONS

- When using **plot**, only one figure window can be up at a time
 - This means that if you have two plot commands in your code, the first plot will be overwritten
 - Additional commands can be used to prevent this
- **figure** – allows multiple figure windows to be open at same time
 - Usage: `figure(#)` above the plot command
- Want multiple plot lines on the same figure?
 - **hold on** – tells MATLAB to keep plotting on the same figure
 - **hold off** – turns off the hold on setting
 - Only need to specify this option once for each figure

MORE PLOT FUNCTIONS

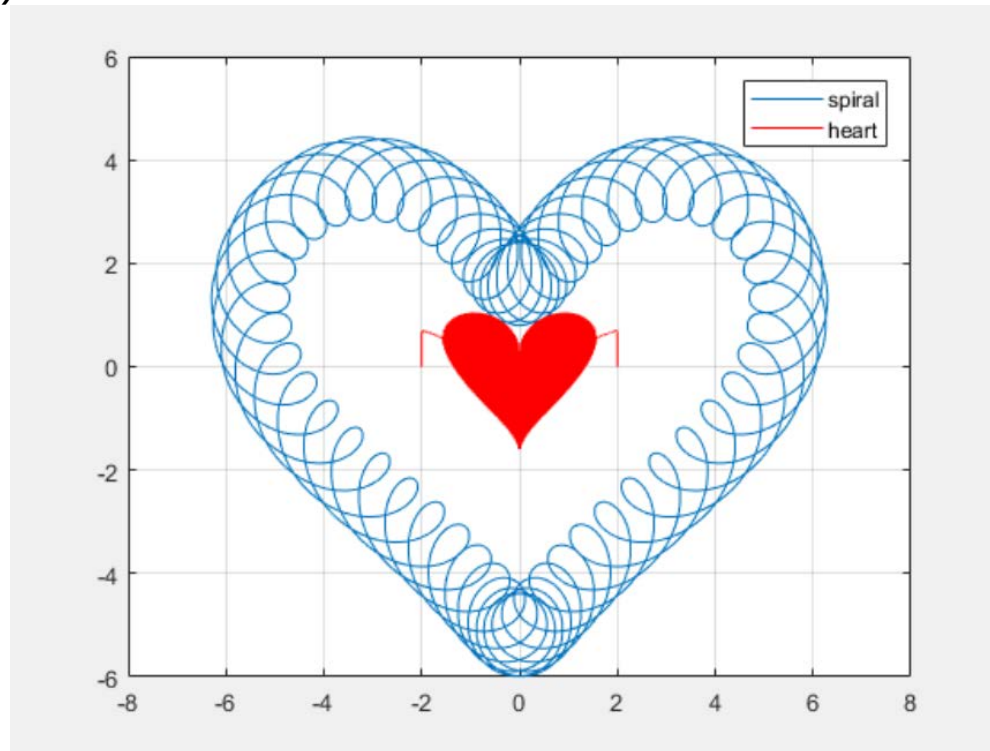
- **legend** – displays color-coded legend for multiple plot lines
 - Usage: **legend('string1','string2',etc)**
 - Strings must be in same order as plots were created
- **grid on** – places grid lines on plot

EXAMPLE

```

t=0:.001:100; %%spiral heart
x=4.5*sin(t)-sin(3*t)+0.8*sin(15.25*t);
y=4*cos(t)-1.5*cos(2*t)-0.6*cos(3*t)-0.3*cos(4*t)+0.8*cos(15.25*t);
plot(x,y)
hold on
x=-2:1/1000:2;    %%heart
y=(sqrt(cos(x)).*cos(200*x)+sqrt(abs(x))-0.7).*(4-x.*x).^0.01;
plot(x,y,'r')
legend('spiral','heart')
grid on

```



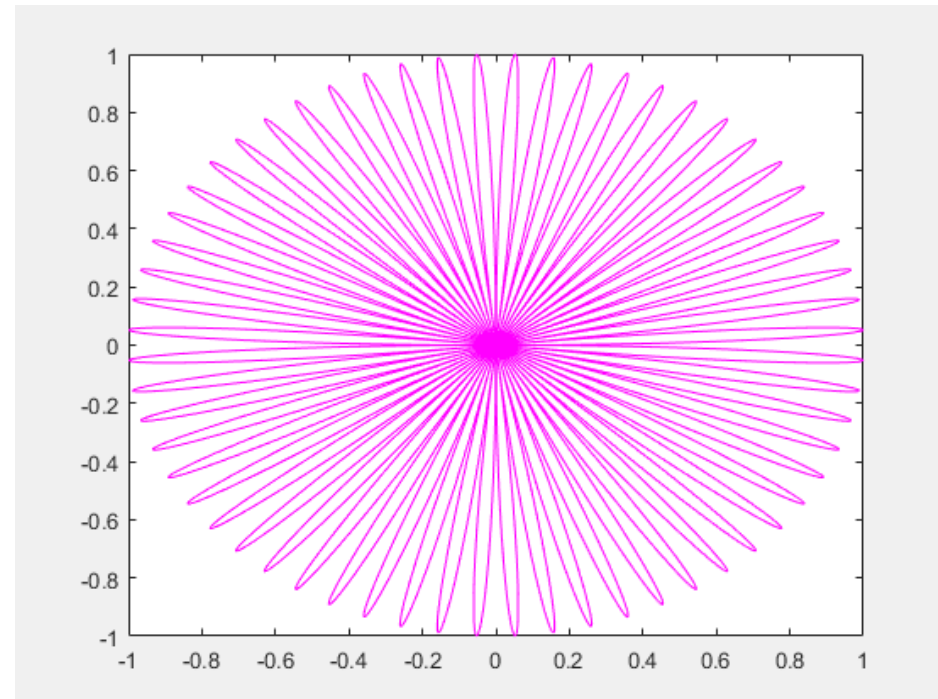
load AND save

- Basic commands to read in data from an external file:
 - **load** – function will load and save data into a matrix with variable name of the filename
 - Usage: **load loaddata.txt**
 - Data will be stored in matrix named loaddata
 - Note: file to be loaded must be located in working directory
 - **save** – function will save matrix data into a file
 - Usage: **save filename.txt variable -ascii**
 - File gets created in the working directory
 - To append to an existing file:
 - **save filename.txt variable -ascii -append**

EXAMPLE

```
t=0:.001:200;  
r=30;  
x=sin(r*t).*cos(t);  
y=sin(r*t).*sin(t);  
p1(:,1)=x;  
p1(:,2)=y;  
save plotdata.txt p1 -ascii  
clear  
load plotdata.txt  
xx=plotdata(:,1);  
yy=plotdata(:,2);  
plot(xx,yy, 'm')
```

Data must be in matrix form



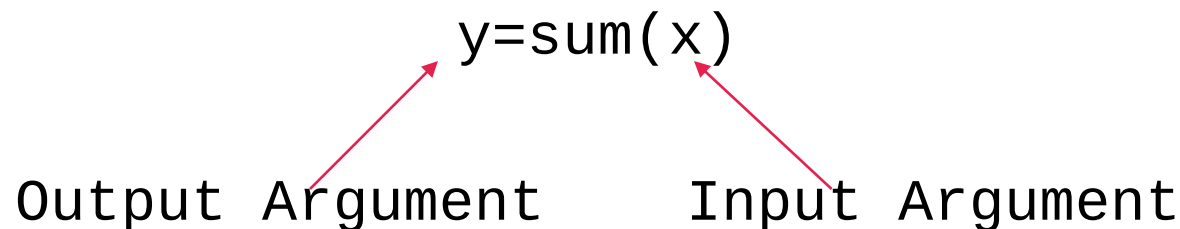
Data saved in variable plotdata

USER-DEFINED FUNCTIONS – ONE OUTPUT

- Think of the function sum
 - Every time you want to sum numbers you simply call this function
 - There's no need to recode the expressions required for the calculation
 - This saves time and leads to cleaner, more efficient coding
- Will now learn how to create your own functions that can be called whenever you need them
- For now, will only look at functions with one output and 1+ inputs

USER-DEFINED FUNCTIONS – TERMINOLOGY

- Let's cover terminology used for functions
- Consider sum again
- When we use sum (or any other function) we
 - **Call** the sum function
 - **Send** an **input argument** (what's in parenthesis to right of sum)
 - Input argument is then used in the function for calculations
 - sum then **returns** an **output argument**



GENERAL FORM OF FUNCTION DEFINITION

- Function Definitions are created in separate .m files
 - Click on New then select Functions
 - A template pops up in new window
- Function Definitions contain the following:
 - Function header (first line)
 - Function body (everything between first and last line)
 - end

FUNCTION DEFINITION

- The header of the function includes several things:

`function` `outarg` = `fname`(`input arguments`)

- `function` - header always starts with the reserved word "function"
- `outarg` - name of the output variable you are calculating in the function
- `fname` - function name
 - should be the same as the name of the m-file in which this is stored
- `input arguments` - correspond to the values that are passed to the function when called
 - The order of the input arguments needs to be the same in the function and in the code that calls the function
 - Input argument names do not have to match but the order does!

EXAMPLE

- Target heart rate calculator
 - $\text{Rate} = (220 - \text{age}) * \text{percent}$
 - User inputs: age, percent (in decimal form)
 - Function THR calculates target heart rate
- Function Definition

```
function [ rate ] = THR( age, percent )
%Calculate target heart rate
rate=(220-age)*percent;
end
```
- Input arguments: age, percent
- Output arguments: rate

EXAMPLE

Script File:

```
age=input('Enter age: \n');  
percent=input('Enter percentage of max heart rate (.65 or .80): \n');  
rate=THR(age,percent);  
fprintf('Target heart rate at %d percent for an age of %d: %d\n',percent*100,age,round(rate))
```

THR.m

```
function [ rate ] = THR( age,percent )  
%Calculate target heart rate  
rate=(220-age)*percent;  
end
```

Command Window Output:

```
Enter age:  
22  
Enter percentage of max heart rate (.65 or .80):  
0.8  
Target heart rate at 80 percent for an age of 22: 158
```

EXAMPLE

- Three ways to call function:
 1. `THR(age, percent)`
 - Result stored in variable `ans`
 2. `rate=THR(age, percent)`
 - Result stored in user-defined variable `rate`
 3. `disp(THR(age, percent))`
 - Result displayed, but not saved
- Note: input and output argument names do not have to match between call statement and function header. Order does have to match.
 - `thr=THR(a, p)` could be used to call function

FUNCTIONS

- Arrays can be passed to functions
- Need to make sure equations are set up properly to handle arrays
 - Use the `.*`, `.\` , `.^` operators in function equations
 - Will allow for calculations with scalars and arrays

PROGRAM

- Format of simple modular program
 - Get inputs
 - User-entered
 - Load data
 - Hard coded values
 - Calculations
 - Call function(s) to perform calculations
 - Note: functions should not print out values calculated
 - Display Results
 - `disp`
 - `fprintf`
 - save data to file

LOCAL VARIABLES

- Functions operate in their own workspaces
- Variables used/created in functions are called **local variables**
- Local variables are only available while function is executing
 - Scope is where variables are available
- Command window and scripts operate in the base workspace
 - Variables created in command window and script files are saved in the base workspace
 - These variables are not available to functions unless they are passed in as input arguments